

Zig-Zag Sparse Attention v0.7 实验报告

方法：

实验比较 dense、local、zigzag_certified、zigzag_certified_cosine、random_regular 和 zigzag_boolean。序列长度固定为 $N_{total}=T=1024$ ， $B=32$ ， $q=32$ ， $d=8$ ，causal=true，seed=0。主方法采用 directed block zig-zag remote paths，并用 unique key + log multiplicity bias 保留重复路径贡献。

canonical graph 只生成一次，所有正式 Copy/WikiText run 都复制并校验同一 graph artifact。random_regular 使用 zigzag actual post-causal per-query K 对齐，正式 run 的 random_k_alignment_error_max=0。

流程：

步骤	说明
生成图	固定 graph_seed=0，生成唯一 canonical graph artifact。输出：outputs/v07_graph_n1024_q32_B32_d8/。
检验图	检查 Rot_G 端口级 rotation map 的双射性；G/H 本身是有向正则图，另检查 doubly stochastic、lambda_G、mu_H、rho、collision、overlap、degree 和 sha256。
跑 Copy main	$N_{total}=1024$ ，copy_source_length=511，六种方法各训练 5000 steps。输出：outputs/copy_v07_main_n1024_q32_B32_d8/。
准备 WikiText	WikiText-103 raw；只用 train split 训练 byte-level BPE tokenizer；tokenize train/test。输出：outputs/wikitext_v07_data_tokenize_n1024/。
跑 WikiText main	读取 Phase 4 固定 tokenizer/tokenized blocks，六种方法各训练 1 epoch。输出：outputs/wikitext_v07_main_n1024_q32_B32_d8/。

任务定义：

Copy：

copy_source_length=(1024-2)/2=511；序列为 [s0,...,s510, SEP, s0,...,s510, EOS]。loss query positions=511..1022，target positions=512..1023。

主要指标是 eval_token_accuracy、eval_loss、eval_sequence_accuracy 和 eval_eos_accuracy。

WikiText：

sequence_length=1024；dataset=Salesforce/wikitext, wikitext-103-raw-v1；

tokenizer_algorithm=byte_level_bpe, vocab_size=32000，min_frequency=2，special tokens=<pad>, <eos>, <unk>。

主要指标是 final_train_loss、test_loss、test_perplexity 和 train_tokens_per_sec。

参数	中文意义	记录位置
N/T	序列长度，本次 1024	selected_graph.json；results.csv 的 N_{total}/T
B/q/d	block 大小、block 数、端口图出度：32/32/8	graph_certificate.json；results.csv
G/H	G 是 $q=32$ 个 block 顶点上的 B-in/B-out regular directed multigraph；本次 $B=32$ ，所以每个 block 入度/出度均为 32。H 是端口图，d-in/d-out regular，本次 $d=8$ 。	selected_graph.json 的 G/H；graph_certificate.json 的 G_type/H_type；degree 诊断字段
Rot_G	不是 G 的顶点映射双射；而是 labelled rotation map $Rot_G: [q] \times [B] \rightarrow [q] \times [B]$ ，把每个出端口 (v,p) 匹配到一个入端口 (w,r)，因此在 $qB=1024$ 个带标签端口上要求双射。	graph_certificate.json 的 rot_g_is_bijection=true；selected_graph.json 的 G.arrival_port/G.permutations
Mxy/log Mxy	重复路径计数与 log multiplicity bias	results.csv 的 multiplicity_mode；graph_certificate 的 collision/unique K；zigzag_budget.json

lambda_G/mu_H/rho	谱证书参数	graph_certificate.json; results.csv 的 lambda_G/mu_H/rho_zigzag_bound
K_eff	causal 后每个 query 对应 key 数	results.csv 的 effective_K_*_after_causal
pairs	causal 后 attention pair 数	results.csv 的 attention_pair_count_after_causal
L-hop/avg_path	Copy shortcut 诊断: 目标是否在 L-hop 内及平均最短路	results.csv 的 target_in_Lhop_rate、average_shortest_path
random same-budget	random_regular 与 zigzag actual post-causal K 对齐	random_budget.json; results.csv 的 random_* 字段

图结构:

项目	值
graph_id	v07_B32_d8_s0_619e9b962fc61bfd
canonical sha256	53ae37a6584833a1d20d51162a01a03d18bf7eeb9fd0efd2ebf3b8a482427a48
N/q/B/d	1024 / 32 / 32 / 8
G/H type	permutation_regular / permutation_regular; G 的 block-level 入/出度为 B=32, H 的端口入/出度为 d=8
raw K/local labelled/remote labelled	96 / 32 / 64
unique total K min/mean/max	71.0 / 79.59375 / 91.0
lambda_G/mu_H	0.0625 / 0.6436248213046418
rho bound/exact	0.9114789606824917 / 0.46386654287979273
certified	True
collision mean	16.40625
remote-local overlap mean	0.0

正式实验设置:

项目	设置
Copy main	steps=5000, batch_size=64, eval_batch_size=64, eval_batches=50, learning_rate=0.001, log/eval every=50, checkpoint_every=500。
WikiText main	train_epochs=1, train_steps=3516, batch_size=16, gradient_accumulation_steps=2, effective_batch_size=32, learning_rate=0.0003, eval=test all。
模型	8 layers, d_model=128, heads=4, ffn_dim=256, dropout=0.1, optimizer=adamw, attention_backend=auto_split。
方法差异	zigzag_certified_cosine 与 zigzag_certified 结构相同，仅 lr_scheduler=cosine; zigzag_boolean 是丢弃 multiplicity 的 boolean 消融。

实验图:

Copy main combined curves

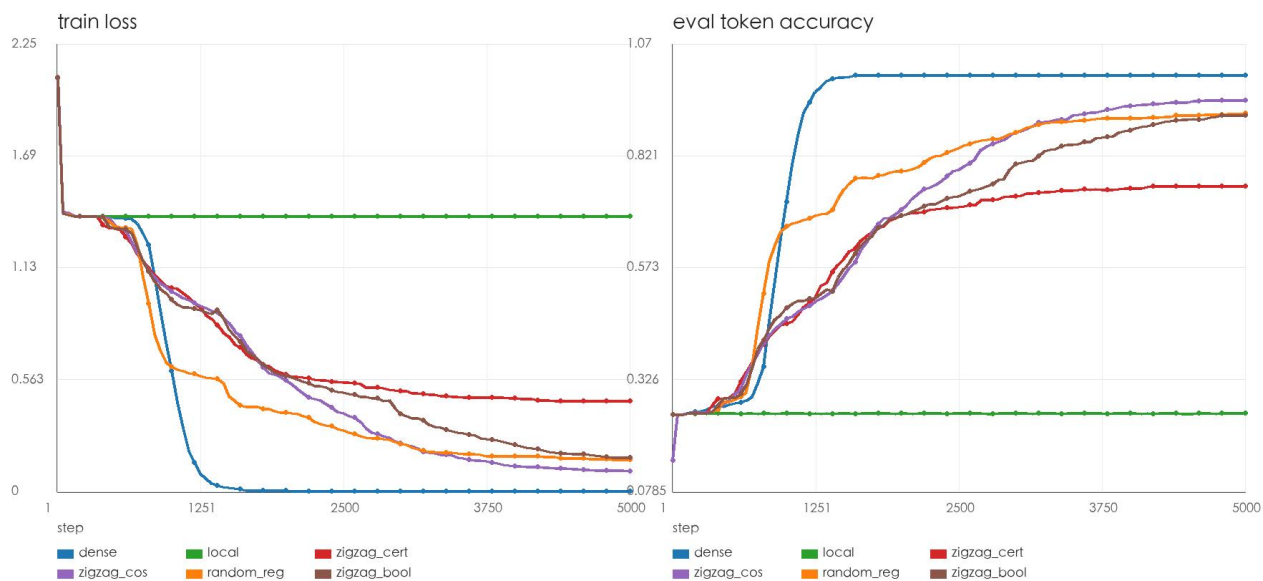


图 1: Copy main 合并曲线, 仅 train loss 与 eval token accuracy。

WikiText main train loss (steps > 200)

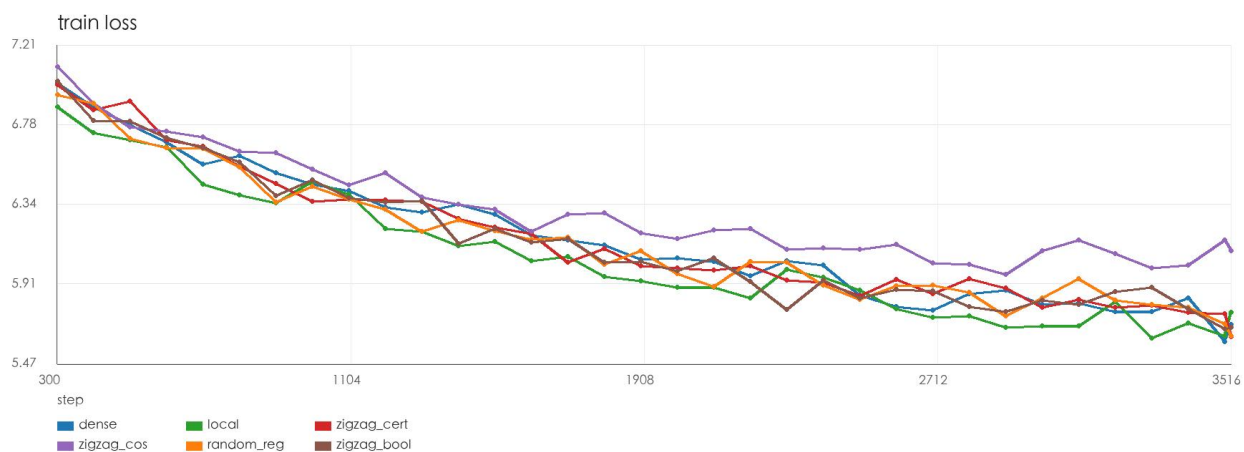


图 2: WikiText main 合并曲线, 仅 train loss。

结果分析:

Copy: dense 达到 token accuracy=1.0, 证明任务和训练可以完全收敛; local 停在约 0.251, 接近四分类随机水平, 说明跨 block 信息必要。zigzag_certified constant 为 0.754, cosine 同结构升至 0.944; random_regular 为 0.915, zigzag_boolean 为 0.911。Copy 单 seed 下, cosine 调度显著改善 zigzag, 但尚不能证明主方法稳定优于 same-budget random/boolean。

WikiText: local 的 test PPL=264.978 最低, dense 为 276.955; random_regular 与 zigzag_boolean 分别为 282.194 和 281.002; zigzag_certified constant 为 287.110, cosine 为 404.143。当前实现下 sparse zigzag 路径速度约 40k train tok/s, 明显低于 local 的 231k tok/s, 说明工程实现还有较大优化空间。

总结:

下表是正式 main run 的关键结果, 每种方法一行。K_eff 表示 causal 后平均每个 query 对应的 key 数, 括号内为 min-max; L-hop rate 和 avg_path 来自 Copy shortcut diagnostics。

method	K_eff	pairs	L-hop	avg_path	Copy token acc	Copy loss	WT train loss	WT test PPL	WT tok/s
dense	512.5	524800	0.998047	1	1	2.32011e-06	5.68609	276.954555	119449.836
local	16.5	16896	0	inf	0.251341	1.383765	5.750624	264.978166	231434.745
zigzag_certified	40.264	41230	0.998047	2.309198	0.754044	0.455293	5.617346	287.11048	41062.769

zigzag_certified_cosine	40.264	41230	0.998047	2.158513	0.944318	0.103008	6.088463	404.142981	40867.155
random_regular	40.264	41230	0.998047	1.95499	0.914902	0.159627	5.623149	282.194177	40614.347
zigzag_boolean	40.264	41230	0.998047	2.309198	0.911292	0.165175	5.669633	281.001867	40808.549

总体判断：v0.7 的 graph artifact、sha 校验、tokenizer/tokenized 数据、random same-budget 对齐和训练日志已经闭环。图结构满足 $\rho_{\text{zigzag_bound}} < 1$ 的认证条件。但从正式结果看，Copy 中 zigzag 对调度敏感，WikiText 中 local 仍是最强基线；下一步应集中在多 seed、随机化 retrieval 任务，以及 sparse attention kernel/数据访问优化。